UNIVERSITÉ LAVAL

Travail pratique N° 1

Cours : GLO-3100	Cryptographie et sécurité informatique
Date limite :	Le 30 octobre 2026 à 23h59

© Honoré H., 2026

Objectif

L'objectif de ce travail est de vous permettre de mettre en pratique les différents systèmes cryptographiques classiques étudiés en classe, d'explorer diverses techniques de cryptanalyse, de vous initier à la stéganographie et de comprendre l'importance du choix d'un mode d'opération approprié.

Il se peut que la description du TP1 présentée ci-dessous manque de précision sur certains aspects. Si c'est le cas, il est de votre responsabilité de le remarquer et de demander l'information qui manque.

Remarques pour l'exercice 1

— Le programme *vigenere_gui.py* sera nécessaire pour l'étape 5. Vous pouvez le télécharger depuis le site web du cours. Cet outil facilitera vos calculs.

— Pour simplifier votre code, vous pouvez utiliser la méthode brute pour inverser une valeur a dans $\mathbb{Z}_n$.

```
int mod_inverse(int a, int n) {
    a = a % n;
    for (int i = 1; i < n; x++) {
        if ((a*i) % n == 1) {
            return i;
        }
    }
}
```

Cependant, en Python, il est plus simple et efficace d'utiliser la fonction intégrée `pow` comme suit : `pow(a, -1, n)`.

Exercice 1 : Désamorcer la bombe (30 points)

Vous êtes partie d'une équipe d'élite en cryptographie, chargée de désamorcer une bombe qui menace de tout anéantir. Le code de désactivation est dissimulé dans une série de messages à chiffrer ou à déchiffrer. Votre mission : les déchiffrer/chiffrer avant que le compte à rebours n'atteigne zéro. Bien que le compte à rebours soit fictif, tentez de relever le défi en moins de 90 minutes pour pimenter l'expérience :).

Vous recevrez cinq messages à chiffrer/déchiffrer, chacun utilisant un type de chiffrement différent. Votre mission sera de chiffrer ou déchiffrer chaque message afin d'obtenir chacun des chiffres du code de désactivation. En combinant les informations de toutes les étapes, vous obtiendrez le code de désactivation final, composé de 5 chiffres.

– Pour l'étape 1, vous devez écrire un programme Python, C/C++ ou Java. Vous ne devez PAS utiliser d'outils en ligne.
– Pour les étapes 2, 3 et 4, vous pouvez choisir de les faire à la main ou écrire un programme Python, C/C++ ou Java. Vous ne devez PAS utiliser d'outils en ligne.
– Pour l'étape 5, vous devez OBLIGATOIREMENT utiliser l'approche vue en classe. Le programme *vigenere_gui.py* facilitera vos calculs. Cet outil vous assistera pour estimer la longueur de la clé, en déduire la clé proprement dite, puis procéder au déchiffrement du message.

Note : Chiffre vs Nombre

Définitions :
- Un **chiffre** est un symbole unique utilisé pour écrire les nombres (0, 1, 2, 3, 4, 5, 6, 7, 8, 9).
- Un **nombre**, en revanche, peut être composé d'un ou plusieurs chiffres (par exemple : 7, 42, 2025).

Exemples :
- Dans « 25 » : il y a 2 chiffres (2 et 5) qui forment un nombre (vingt-cinq)
- Dans « 2023 » : il y a 4 chiffres (2, 0, 2, 3) qui forment un nombre (deux mille vingt-trois)

Dans cet exercice : Quand nous parlons du « premier chiffre du code de désactivation », nous faisons référence à un chiffre unique (0-9), pas à un nombre à plusieurs chiffres.

Tentez la chance de gagner un petit cadeau (facultatif)

Soyez le premier à envoyer le bon code de désactivation à *olufemi-honore-etsmoberg.hounwanou@ift.ulaval.ca* et vous recevrez un petit cadeau. Il est impératif de ne **PAS** avoir utilisé d'outils en ligne !

Étape 1 : Chiffrement à décalage (2.5 points)

Le premier message est chiffré en utilisant un chiffrement à décalage. Déchiffrez le message en déterminant le bon décalage. Le message déchiffré contient un indice pour le premier chiffre du code de désactivation.

Message chiffré 1 :

```
WPZIJ NYDVN EZIZN VDNKV NNDOP KVMGZ NAMVI XVDNJ PZNKV BIJGH VDNXZ LPZEZ NVDNX ZNOLP ZOPQZ
PSJWO ZIDMP IZWJI IZIJO ZVXZK MZHDZ MOMVQ VDGKM VODLP ZGZNI JOZNN JIODH KJMOV IOZNX ZMOZN
HVDNX ZIZNO KVNXZ LPDXJ HKOZG ZKGPN GZNNZ IODZG XZNOY ZKMZI YMZKG VDNDM VXZLP ZOPAV DNZIO
JPOXV NHJDE VDYPA PIVZX MDMZX ZOKCD CDVGJ MNKMZ IYNPI ZBMVI YZDIN KDMVO DJIXV MOPZI VPMVN
WZNJD IKJPM YZXCD AAMZM GZKMJ XCVDI HZNNV BZOMJ DNNZX JIYZN YZQMV DZION PAADM Z
```

Étape 2 : Chiffrement affine (2.5 points)

Le deuxième message est chiffré en utilisant un chiffrement affine. Le message déchiffré fournit un indice sur le second chiffre du code de désactivation.

Message chiffré 2 :

```
VWBGZ BITZS ATKVV GGRET VWTSV UTATL QVUUV GTPPT ISWTZ HYBZT ZEBIS ZTHBG ZTGVL MLGTS VPTZL
GTNLT SLVEV IHTGV ZAVIZ WTZTS VUTZL IUTLH BPPTW TIRET VLATA RMMRH LWSTA TZSUA THTHB LGZAV
RWWTL GZRWJ VHBPO RTIAT SVUTZ AVIZH TUGTP RTGTQ TGHRH T
```

Étape 3 : Chiffrement de Hill (5 points)

Le troisième message est : *JAMAIS TROIS SANS QUATRE HAHAX*. Vous devez chiffrer ce message en utilisant le chiffrement de Hill. La matrice clé est :

$$\begin{bmatrix} 3 & 4 \\ 6 & 3 \end{bmatrix}$$

Le message chiffré cache un mot de 4 lettres qui correspond au troisième chiffre du code de désactivation.
Indice : Le message chiffré commence par **BKKWC...**

Étape 4 : Carré de Polybe (5 points)

Le quatrième message est : *La fin est proche. Ce n'est pas le moment d'abandonner*. Vous devez chiffrer ce message en utilisant le carré de Polybe. Le mot de passe est *CRYPTGLO*. Le chiffre le plus fréquent dans le message chiffré correspond au quatrième chiffre du code de désactivation. En cas d'égalité entre plusieurs chiffres, faites-en la somme.

Étape 5 : Chiffrement de Vigenère (15 points)

Pour cette étape, vous devez obligatoirement utiliser l'approche vue en classe. Le programme *vigenere_gui.py* facilitera vos calculs. Il vous assistera pour estimer la longueur de la clé, en déduire la clé proprement dite, puis procéder au déchiffrement du message.

Le dernier message a été chiffré à l'aide du chiffrement de Vigenère. Déchiffrez-le pour révéler le dernier chiffre du code de désactivation. Ce chiffre correspond à la somme de tous les chiffres présents dans le texte, répétée de manière récursive jusqu'à obtenir un seul chiffre.

Message chiffré 5 :

```
WCEFZ WSGTU KYFVS DAMBQ ENAQC DEAZS IAEPX MGTEF RMUEE FPEWG RVYFS RUPUY DUECR EWJIE GVSUI TKVQV
TRTSB TERZN SJPRX IELEI UYGVN NAVSQ AHZEB KAQKG VZFSX IFVTP NMTWR RXPSJ DVLJS IEAZW AVSFG KSJJR
YYWJV EGMAV NGLMS IDRBS IJJNP SIKEF OQDCE ZKRHH URRUI VSYOK BVSCU YFRLY URUVR YKXSO TRSEW JJRZM
SESNR IFVPR ZIFAE FAMGM RNOQS ETSOI FUEIU YGAEY KWCLL VMRSR NBAZS RUCUY FHURS SBDEF YEUVS BOXPZ
EAIPO ZRBAM XVSHO WTZEE JIJFU FRIGX AEYIH CEFMS G
```

Livrables

Un dossier nommé `Exercice_1` contenant :

1. Un document PDF nommé `Rapport_Exercice1_[VotreNom].pdf` détaillant les étapes que vous avez suivies pour chiffrer/déchiffrer chaque message.
2. Les codes sources des programmes que vous avez développés.
3. Le code de désactivation final et une explication de la façon dont vous l'avez dérivé (facultatif).

Exercice 2 : Stéganographie et reconstruction de code (25 points)

Dans cet exercice, vous explorerez la stéganographie en cachant des morceaux d'un script Python dans des fichiers image sur l'ordinateur d'une victime fictive. Vous développerez ensuite un script pour localiser ces images, extraire le code caché, reconstruire le script original et l'exécuter. Cet exercice approfondira votre compréhension des techniques de dissimulation de données et de leurs implications en matière de sécurité.

Aperçu général du travail à faire :

1. **Préparation**
   - Vous commencerez avec un script Python fourni.
   - Divisez le script en plusieurs morceaux.
2. **Cacher les morceaux de code**
   - Intégrez chaque morceau dans des fichiers image séparés en utilisant des techniques de stéganographie.
   - Assurez-vous que les images semblent inchangées lors d'une inspection occasionnelle.
3. **Simulation de l'ordinateur de la victime**
   - Créez une structure de répertoires imitant le système de fichiers typique d'un utilisateur.
   - Distribuez les images stéganographiées dans cette structure.
4. **Reconstruction et exécution du script**
   - Développez un script Python pour :
     - Rechercher et identifier les images stéganographiées.
     - Extraire les morceaux de code cachés.
     - Réassembler le script original.
     - Exécuter le script reconstruit.
5. **Documentation et vidéo de démonstration**
   - Rédigez un rapport détaillant votre méthodologie, les défis rencontrés et les implications de la stéganographie en matière de sécurité.
   - Enregistrez une courte vidéo démontrant que votre solution fonctionne correctement.

Instructions détaillées

1. Préparation

- **Obtenir le script :**
  - Téléchargez le fichier `script_secret.py` fourni sur le portail du cours.
- **Diviser le script (`decouper_script.py`) :**
  - Divisez le script en N parties, en veillant à ce qu'il y ait au moins 3 parties.
  - Sauvegardez chaque partie sous `partie_1.txt`, `partie_2.txt`, ..., `partie_N.txt`.

2. Cacher les morceaux de code

- **Sélectionner des images porteuses :**
  - Choisissez N fichiers image (par exemple, formats `.png` ou `.jpg`).
- **Intégrer les morceaux de code (`cacher_morceaux_dans_images.py`) :**
  - Utilisez une librairie de stéganographie ou écrivez un script personnalisé pour intégrer chaque morceau de code dans une image.
  - Assurez-vous :
    - Que la qualité de l'image reste élevée.
    - Que les données cachées ne soient pas détectables par une analyse simple.

3. Simulation de l'ordinateur de la victime

- **Créer la structure de répertoires :**
  - Simulez un environnement utilisateur typique avec des dossiers tels que `Documents`, `Images`, `Téléchargements`, etc. Voici un exemple :

```
Documents/
    resume.docx
    notes.txt
Images/
    vacances/
        toronto.jpg
        paris.png
    famille/
        papa.png
        maman.png
        noel.jpg
Téléchargements/
    software.exe
    wallpaper.png
```

- **Distribuer les images stéganographiées :**
  - Placez chaque image stéganographiée à différents emplacements dans la structure de répertoires.

4. Reconstruction et exécution du script

- **Développer le script d'extraction (`extraire_et_executer.py`) :**
  - **Recherche :**
    - Parcourez la structure de répertoires pour trouver les fichiers image.
  - **Extraction :**
    - Identifiez les images contenant des données cachées.
    - Extrayez les morceaux de code de ces images.
  - **Reconstruction :**
    - Réassemblez les morceaux de code dans le bon ordre pour reconstruire `script_secret.py`.
  - **Exécution :**
    - Exécutez le script reconstruit depuis votre programme.

5. Documentation et vidéo de démonstration

- **Rapport écrit :**

- Rédigez un rapport incluant :
  - **Introduction :** Brève présentation de la stéganographie et de sa pertinence.
  - **Méthodologie :** Expliquez les techniques utilisées pour l'intégration et l'extraction des données, et comment vous avez assuré la discrétion des données cachées.
  - **Défis et solutions :** Décrivez les problèmes rencontrés et comment vous les avez résolus.
  - **Implications en matière de sécurité :** Analysez les risques potentiels et comment de telles techniques peuvent être détectées ou empêchées.
  - **Conclusion :** Résumez vos découvertes et réflexions sur l'exercice.
- **Vidéo de démonstration :**
  - Enregistrez une courte vidéo (2 à 10 minutes) démontrant que votre solution fonctionne correctement.
  - La vidéo doit inclure :
    - L'exécution de votre script d'extraction.
    - La démonstration de l'extraction des morceaux de code.
    - La reconstruction du script original.
    - L'exécution réussie du script reconstruit.
  - Assurez-vous que la qualité vidéo est suffisante pour lire le code et les sorties affichées.

Livrables

Un dossier nommé `Exercice_2` contenant :

1. **Fichiers de code :**
   - `decouper_script.py`
   - `cacher_morceaux_dans_images.py`
   - `extraire_et_executer.py`
   - Tous les autres scripts ou outils développés pour l'intégration/l'extraction.
2. **Images stéganographiées :**
   - Tous les fichiers image contenant les morceaux de code cachés.
3. **Rapport :**
   - Un document PDF nommé `Rapport_Exercice2_[VotreNom].pdf`.
4. **Vidéo de démonstration :**
   - Une courte vidéo (format MP4 ou AVI) nommée `Video_Demonstration_[VotreNom].mp4` montrant que votre solution fonctionne correctement.

Critères d'évaluation

- **Fonctionnalité (10 points) :**
  - Succès de l'intégration et de l'extraction des morceaux de code.
  - Reconstruction et exécution correctes du script original.
  - Efficacité de la technique de stéganographie.
  - Innovation dans les méthodes de dissimulation et d'évasion de la détection.
- **Qualité de la documentation (7.5 points) :**
  - Clarté, organisation et exhaustivité du rapport.
  - Profondeur de l'analyse concernant les implications en matière de sécurité.
- **Qualité de la vidéo (5 points) :**
  - La vidéo démontre clairement le bon fonctionnement de la solution.
  - Bonne qualité audio et vidéo.
- **Qualité du code (2.5 points) :**
  - Lisibilité et organisation du code.
  - Commentaires et documentation adéquats.

Exercice 3 : Les modes d'opération, en vrai (15 points)

Pour cette partie, vous **devez** utiliser l'implémentation AES-128 d'une bibliothèque standard (`cryptography` en Python, `Crypto++` en C++, `javax.crypto` en Java). On n'implémente jamais soi-même une primitive cryptographique (module 3a !).

1. **L'expérience du pingouin.** Chiffrez l'image `image_secrete.bmp` avec AES-128, une première fois en mode ECB et une seconde fois en mode CBC.

   Conservez en clair les 54 premiers octets du fichier BMP et chiffrez uniquement les octets suivants. Utilisez un remplissage **PKCS#7** lorsque cela est nécessaire. Pour le mode CBC, utilisez un vecteur d'initialisation (IV) de 16 octets généré aléatoirement.

   Reconstituez ensuite deux fichiers BMP contenant l'en-tête original suivi des données chiffrées, puis ouvrez-les dans une visionneuse d'images.

   Livrez les fichiers `image_ecb.bmp` et `image_cbc.bmp`, ainsi que votre programme. Expliquez, en vous appuyant sur la définition du mode ECB, pourquoi l'une des deux images révèle encore une partie de la structure de l'image originale. (8 pts)
2. **Défi.** Le fichier `defi.txt` contient deux textes chiffrés, encodés en hexadécimal, obtenus en appliquant un XOR avec exactement le même flux de clé.

   En exploitant

   $$c_1 \oplus c_2 = m_1 \oplus m_2$$

   et le fait que les deux messages sont rédigés en français, utilisez la technique du *crib dragging* avec des fragments probables tels que « `le` », « `que` », etc., afin de retrouver les deux messages en clair.

   Une attaque semi-automatique est acceptée : votre programme peut proposer les positions ou les fragments plausibles, mais vous pouvez sélectionner manuellement les cribs pertinents.

   Livrez les deux messages retrouvés, votre programme ainsi qu'une courte explication de votre démarche. (7 pts)

Livrables

Remettez un dossier nommé `Exercice_3` contenant un rapport PDF intitulé `Rapport_Exercice3_[VotreNom].pdf`.

Le rapport doit contenir :

— le mode d'emploi des outils développés ou utilisés ;
— les deux textes clairs retrouvés dans le défi ;
— les deux images obtenues avec les modes ECB et CBC ;
— une explication de la différence observée entre les deux images ;
— un court retour d'expérience indiquant ce qui a été le plus difficile et pourquoi.

Remarques

1. Ce TP est à effectuer de **façon individuelle**.
2. Les langages Python, C/C++ ou Java sont permis.
3. Assurez-vous que votre code inclut des instructions sur la façon de l'exécuter.
4. Attention au plagiat ! Faites vos TPs par vous-même.

Retards : Le 30 octobre 2026 à 23h59. Les travaux pratiques doivent être impérativement remis via le portail de cours. Aucune remise par courriel n'est acceptée. Un travail qui ne respecte pas les directives de remise se verra pénalisé de 20%. Tout travail remis en retard se verra pénalisé de 25% par jour de retard. Chaque journée de retard débute dès la limite de remise dépassée (dès la première minute). Un retard excédant 2 jours provoquera le rejet du travail pour la correction et la note de 0 pour ce travail.

Remises multiples : Il vous est possible de remettre votre TP plusieurs fois sur le portail des cours. La dernière version sera considérée pour la correction.

À remettre

Utiliser le site web du cours pour remettre un seul fichier ".zip" (de taille maximale 250 Mo) qui porte votre nom au complet et qui contient un répertoire par exercice. Quand il s'agit d'un exercice de programmation, le répertoire en question doit contenir le code source **bien commenté**. Dans les rapports PDF, les réponses doivent conserver les mêmes numéros que les questions, et les captures d'écran (si nécessaire) doivent être bien lisibles.

Utilisation de l'intelligence artificielle (IA) générative non autorisée

L'utilisation de systèmes d'IA n'est **PAS** autorisée dans cette évaluation, car elle nuirait à l'évaluation de votre réelle atteinte des objectifs du cours.

- Il est ***interdit*** **d'inclure des extraits du matériel de cours, incluant un copier-coller d'un énoncé de travail pratique**, dans des requêtes soumises à un système d'« IA générative » (tels que ChatGPT, Claude, etc).
- Il est ***interdit*** **d'utiliser, dans la solution d'un travail pratique, tout extrait de code généré par un système d'IA** (tels que ChatGPT, Claude, etc) ; cela sera considéré comme du plagiat.
- Il est ***permis*** d'utiliser ChatGPT (ou un système équivalent) comme un **outil de référence pour vous aider dans votre apprentissage**, en faisant l'effort intellectuel de formuler des requêtes de votre propre cru et de rédiger par vous-mêmes les solutions des travaux pratiques remis.

Le non-respect de ces exigences constitue une infraction relative aux études et pourrait mener à l'application des sanctions prévues au Règlement disciplinaire.

Bonne chance !